Assignment 7: API Design and Implementation

The purpose of this assignment is to design an API with a moderate level of complexity, using software design tools provided by the Express framework and intermediate SQL query techniques.

The first part of this assignment involves refactoring the code you wrote for **Assignment 5: Introduction to Relational Databases.** Make a copy of the code for that assignment and use it as a starting point for this assignment.

Task 1: Improving the Software Design

Your first task is to modify your API program to use a better program architecture. After completing this task, your API should work the same as it did before, with a few changes:

- To simplify things, you can remove the functionality for **adding new ratings** and **getting the average rating for a retrieved book.**
- Rather than using the ISBN number as a unique identifier for books, instead use the book's **id.** (This makes more sense, since the **id** is the primary key of the **books** table.)

Refactor your program to include the following:

- A **routes** folder, with the following files, to implement request routing logic:
    - **index.js, authorsRouter.js, booksRouter.js** (no routing file for **ratings** is required, as we're removing that functionality.)
- A **controllers** folder, with the following files, to implement the request and response logic:
    - **authorsController.js, booksController.js**
- A **models** folder, with the following files, to implement the database query logic:
    - **authorsModel.js, booksModel.js**
        - To start, simply copy the SQL query logic from Assignment 5.

Test your API to make sure it still works! While you're at it, you can delete the form in **index.html** for creating new ratings. Also, change the **Details for Selected Books** section of **index.html** to use **id** numbers instead of **ISBN** numbers. Use the powers of 2 from **2 to 32** as the **id** numbers.

As you work through the assignment, make sure that each part of the application—**routes, controllers,** and **models**—stay in their lane and don't infringe on each others' responsibilities!

Task 2: Improving the SQL Queries

In Assignment 5, we responded to a request for an individual book by sending an object that contained the name of the book's publisher. This required an extra query because each book refers to its publisher by **publisher_id,** and the names of the publishers are stored separately in another table called **publishers.** It turns out there's a way to do this with a single query using something called a **join.** Joins allow us to fetch data from other tables using **foreign keys** and include it in the result set. Here's the syntax for performing a simple join:

***SELECT** tableA.columnAName, tableB.columnBName, ... **FROM** tableA*
***JOIN** tableB **ON** tableA.foreignKey = tableB.primaryKey;*

The above syntax will include data from **tableB** where the **foreign key** of **tableA** matches up with a **primary key** from **tableB.**

in **booksModel,** rewrite the query for getting a single book using the **JOIN** command.

Task 3: Fleshing Out the API

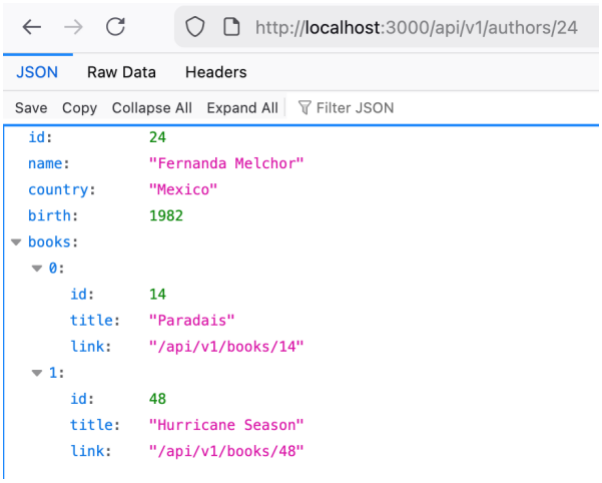
The original API we made in Assignment 5 didn't have an endpoint for getting details about a single author. Let's add that functionality for the path **/api/v1/authors/:id**

The response sent should include a list of books written by the author. Interestingly, the **authors** and **books** tables are connected by a **junction** table called **authored** that links the ID of each other to the IDs of the books they've written. This junction table is necessary to mediate the **many-to-many** relationship between **books** and **authors.**

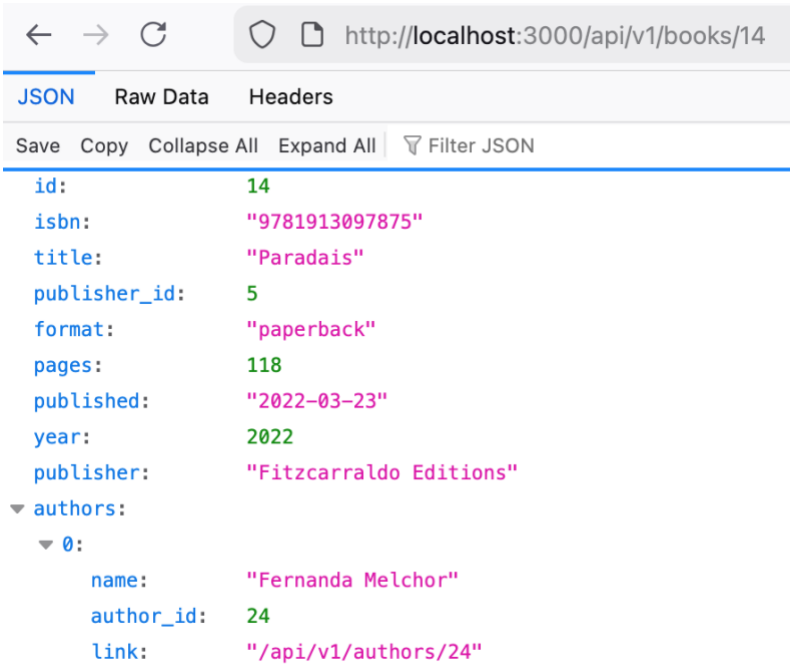
There are a few ways to do this: you can perform **two** queries with a **single join each**, or one query with **two joins.** Which you choose is up to you. (As a challenge, you might want to try to figure out how to do two joins in a single query.)

The list of books should be represented in the author object as an array, with an object representing each book. For each book, add a property called **link** that refers to the **URL path** of that book. For example **/api/v1/books/4**

In the end, the response sent to the client should look something like this:



Next, for the **GET /books/:id** endpoint, use a similar technique to get an array of authors associated with the book, each with a **link** property. A typical response should look like this:



In **index.html,** add a section called **Details for Selected Authors** that has a list of hyperlinks to authors with IDs that are multiples of 6, from 6 to 30.

Task 4: Reviews Table

In Assignment 5, we used our database's **Ratings** table to implement some logic, but in this assignment we're going to bypass that table entirely and create a new data entity: **Reviews!**

Reviews are kind of like ratings—with a numerical score between 0 and 5—but they also have a user's name and a text-based comment. (And a unique ID which, weirdly, the **ratings** table doesn't have.)

Create a file called **db-setup.sql** in the root of your project folder; this SQL script should do two things: drop any previously-existing **reviews** table, and create a new **reviews** table with the necessary columns, keys, and constraints. Here's a rough template to get you going:

DROP TABLE IF EXISTS reviews;

CREATE TABLE reviews (
-- set up columns and constraints here
);

This table should have the following columns:

- **id**, an **integer**
- **reviewer,** a **text** column represent the name of the reviewer;
- **rating**, an **integer** column representing a rating from 1 to 5;
- **book_id**, the **id** of the book this review pertains to;
- **comment,** a **text** column with the reviewer's comments about the movie.

The **id** column should be the **primary key**. You can set that up by adding the following to your CREATE TABLE query:

PRIMARY KEY (id),

The **book_id** should have a **foreign key** constraint, which means the DBMS recognizes this column as a foreign key and treats it carefully. You can set that up by adding this:

FOREIGN KEY (book_id) REFERENCES books (id),

You have inferred that the syntax for a foreign key constraint is like this:

FOREIGN KEY (fk_column_in_this_table) REFERENCES other_table (pk_of_other_table)

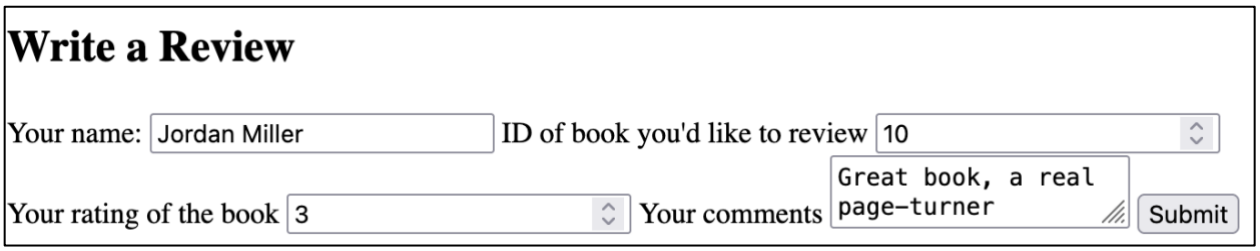
Finally, let's add a **CHECK** constraint that ensures values in the **rating** column are actually between 0 and 5. You can do that by adding this code:

CONSTRAINT rating_range CHECK (rating >= 0 AND rating <= 5)

Run the **db-setup.sql** script and check the **longlist-advance.db,** using **SQLite Viewer**, to make sure the table is being set up properly. (It won't show the CHECK constraint so you'll have to test that later.)

Task 5: Reviews Endpoints

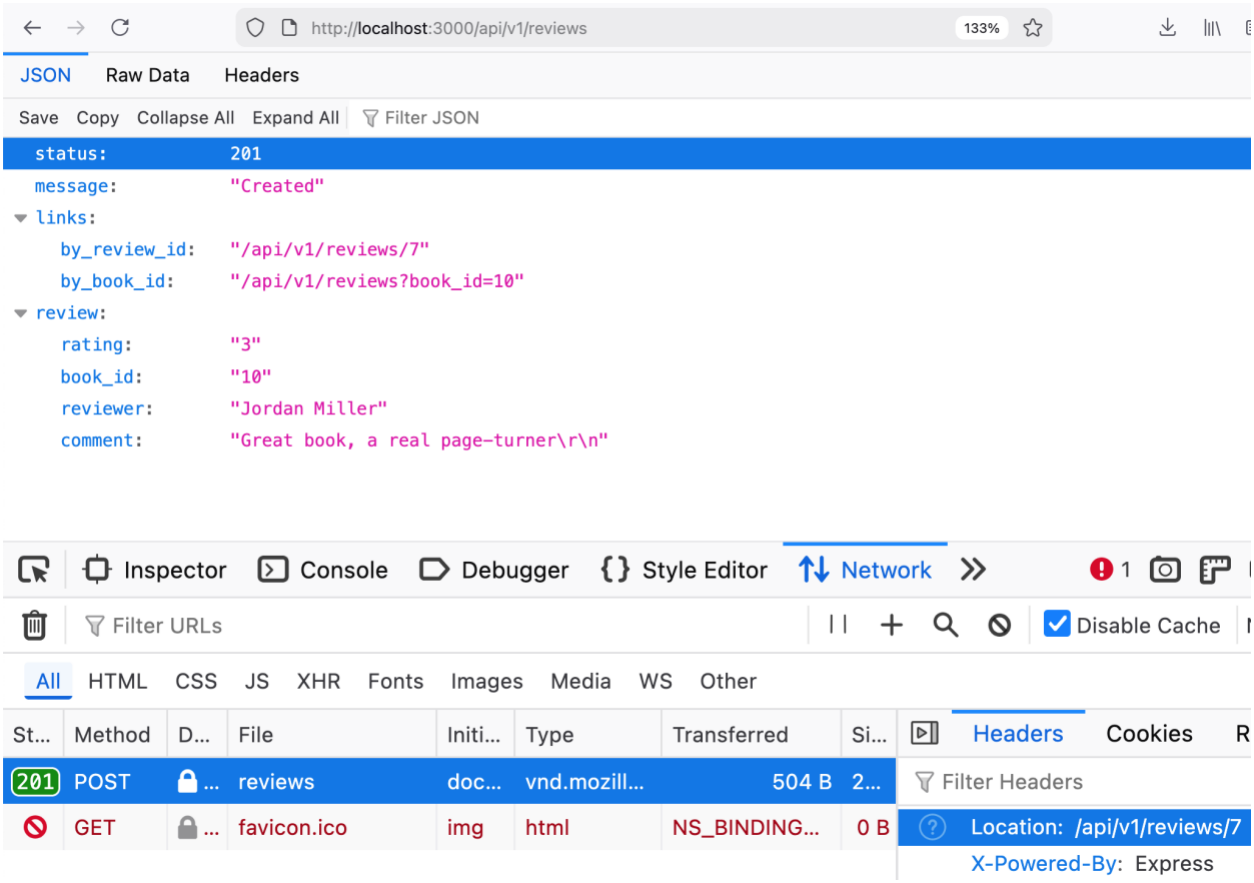
Now let's add a few API endpoints to expose this new table for clients. But first, make a **form** in **index.html** for submitting new **reviews.** When rendered in the browser, it should look something like this:



Now create a route for **POST /api/v1/reviews** that inserts a new **review** into the database. Upon successfully creating a new review, your API should send a response with the following:

- a **201** response status code (use **res.status()** to set this)
- a **Location** HTTP response header with the path of the newly-created resource. Use **res.set('Location', <value>)** to set this. The format of the path should be **/api/v1/reviews/id.** This header is conventional with the HTTP protocol, and clients may expect it.
- A response **body** with the following:
    - The response status (201)
    - A short text message explaining the response
    - An object called **links** with two properties:
        - **by_review_id:** the URL path to this new review
        - **by_book_id:** the URL path for retrieving all reviews associated with a particular book. The format of the path should be **/api/v1/reviews?book_id=<book ID>**
    - an object called **review** with the same data that has just been stored in the database. (You don't have to query it from the database, you can build it from the POST request.)

Here's what a response should look like, with the Network tab of the developer tools to show the response details:



Now create a route for **GET /api/v1/reviews/:id** to get details of a single review. (You don't have to create the endpoint for getting a collection of reviews by book_id, although you can if you'd like to get more practice.)

In **index.html**, create a section called **Selected Reviews by ID** with a list of hyperlinks to a few reviews in the database.

Task 6: Error Handling

Even a simple API like ours is complex and takes a lot of time to build, so we're going to simplify the task by NOT implementing all the necessary data validation and error handling. Of course, in real life, all input from the user (like route parameters, query parameters, request bodies, and anything else) must be validated and sanitized, and our application must respond to error situations in sensible, helpful, and secure ways.

To get a feel for what this looks like, we'll implement just one type of error handling, for **GET /books/id**, in the case that a client requests the ID of a book that doesn't exist.

First of all, figure out how to determine, using the result of the SQL query, whether a book was successfully retrieved or not. If no book was retrieved, do the following:

- Create a response **body** with the following:
    - The response status (which will be 404 for "not found")
    - A response message (something like "A book with that ID was not found")
- Send the response object with a **404** status (again, using **res.status()**)

Optionally, for practice, you can try the following:

- Validate the data from the form; if it's not valid, send a response with a **422** status (unprocessable entity) and a response body explaining the validation failures;
- Send sensible responses when the client tries to access authors or reviews that don't exist.

Note that requesting a collection (like all books) and not getting anything (maybe because the client has specified a non-existent format or page) is NOT an error; it's a successful request that retrieved an empty set.

[OPTIONAL] Task 7: API Testing and Documentation with Postman

Your next and final task is document and test your API using the **Postman** service.

Create a **collection** for your API, and a **request** for each of your API's endpoints; for each endpoint, add appropriate documentation for:

- its overall purpose
- The route parameters (Postman calls these *path parameters*)

Export your Postman collection as a JSON file by doing the following: in your Postman workspace, beside the name of your collection, click the **...** to open a menu, then navigate to **More,** then **Export,** then **Export JSON.** Name this file **api-documentation.json** and place it in the root of your project folder.

Checklist

- Task 1
  - API successfully refactored using routes, controllers, and models folders;
- Task 2
  - JOIN used to connect each book with its publisher;
- Task 3
  - JOIN used to connect each author with their books, and each book with their authors
  - Links created for book and author responses;
- Task 4
  - Reviews table created with necessary columns, PK, FK, and CHECK constraints
- Task 5
  - Form for creating new review
  - POST for creating a new review with necessary response attributes
  - Endpoint for getting a review
- Task 6
  - 404 error handling for /books/id
- Task 7
  - Optional: Postman collection and requests created, with documentation, and exported

Hand In

Make sure you project folder does not contain any unnecessary files, like these instructions, the node_modules folder, etc. Rename the folder to **a7-firstname-lastname**, zip it, and hand it in to Brightspace. Ask me to check your work to receive a grade.